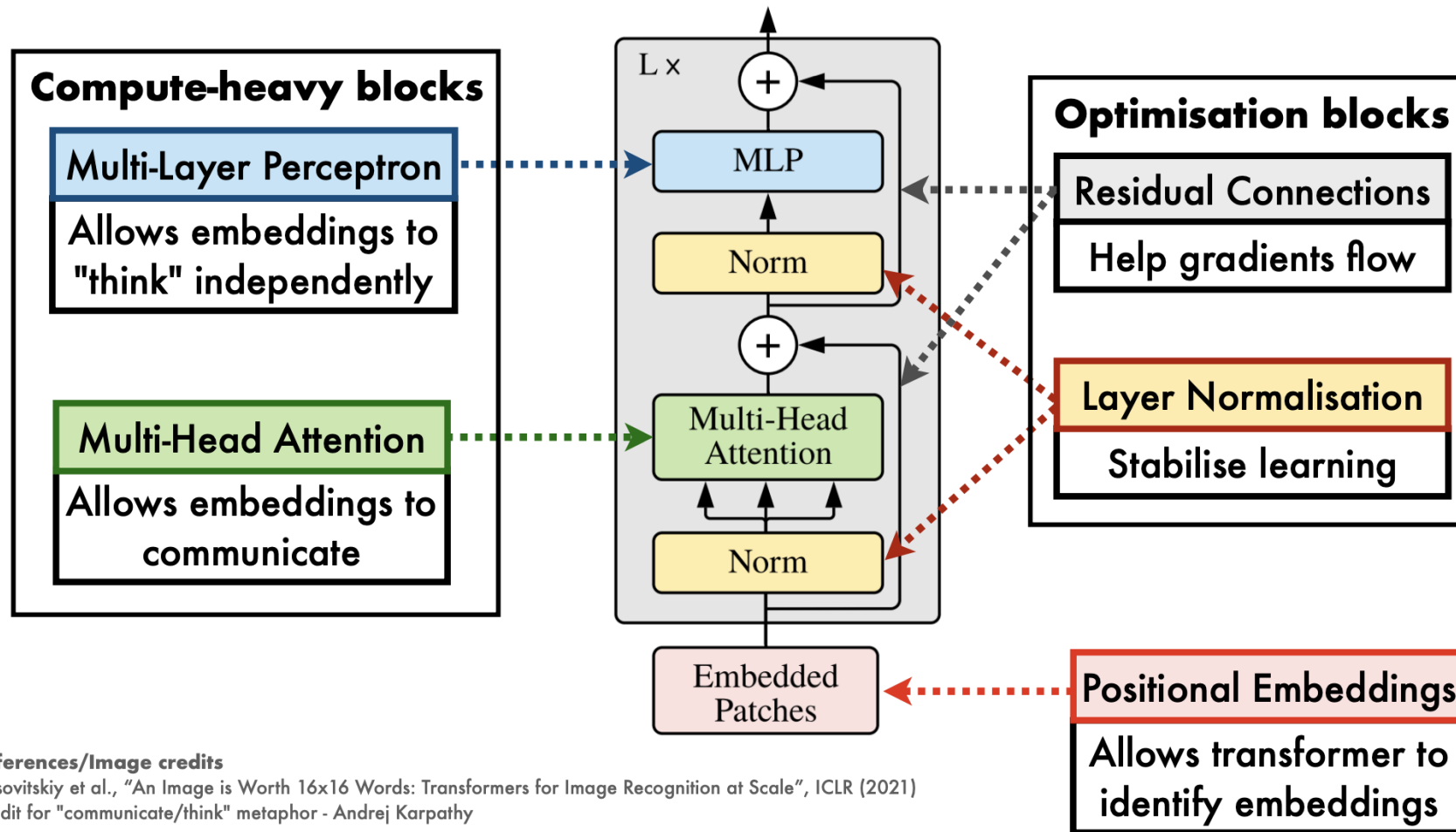


Transformers For All

Transformers

Transformer Encoder

Five key ideas

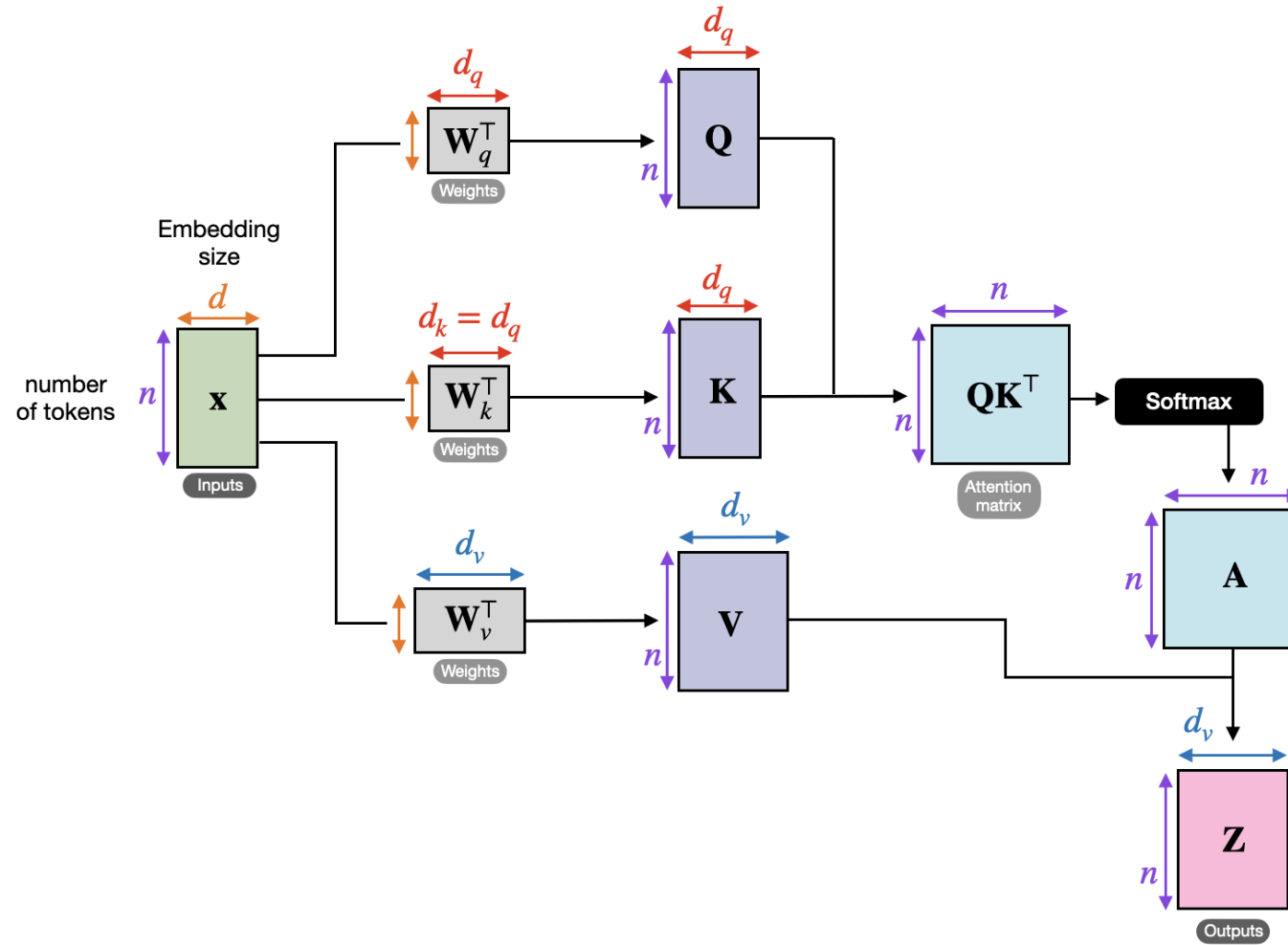


References/Image credits

Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR (2021)

Credit for "communicate/think" metaphor - Andrej Karpathy

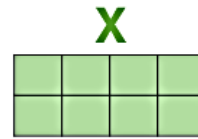
Self Attention: Computation



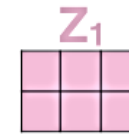
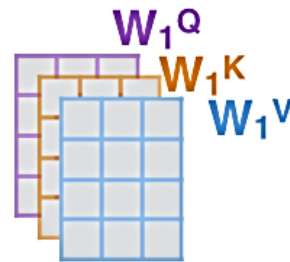
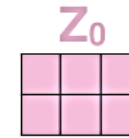
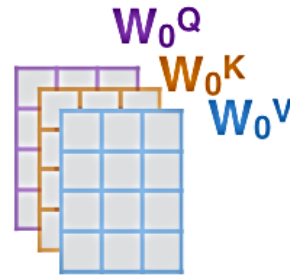
Multi-Head Attention

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking
Machines



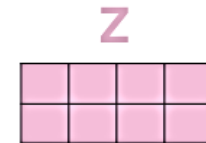
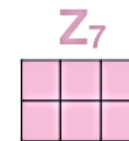
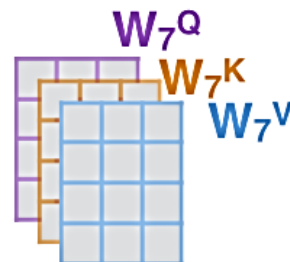
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



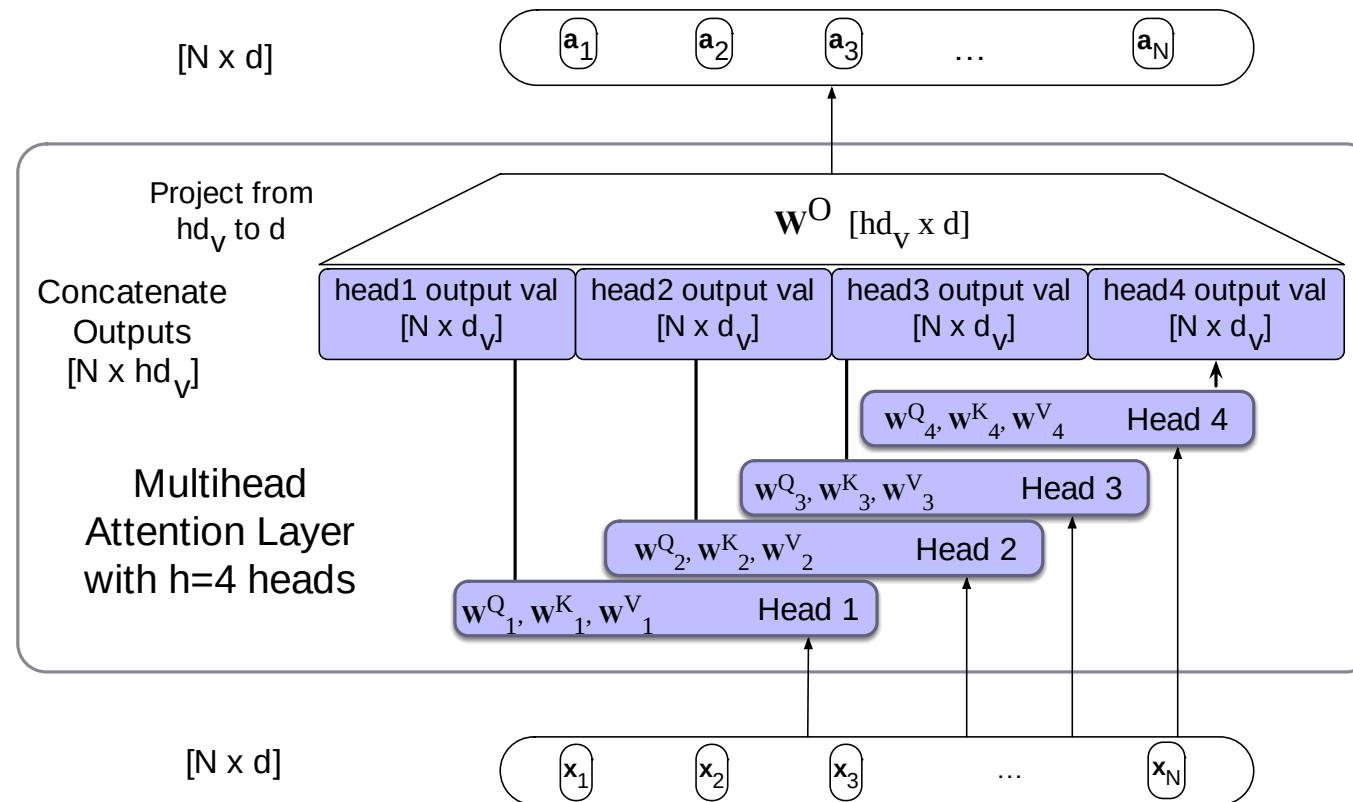
...

...

...



Multi Head Attention

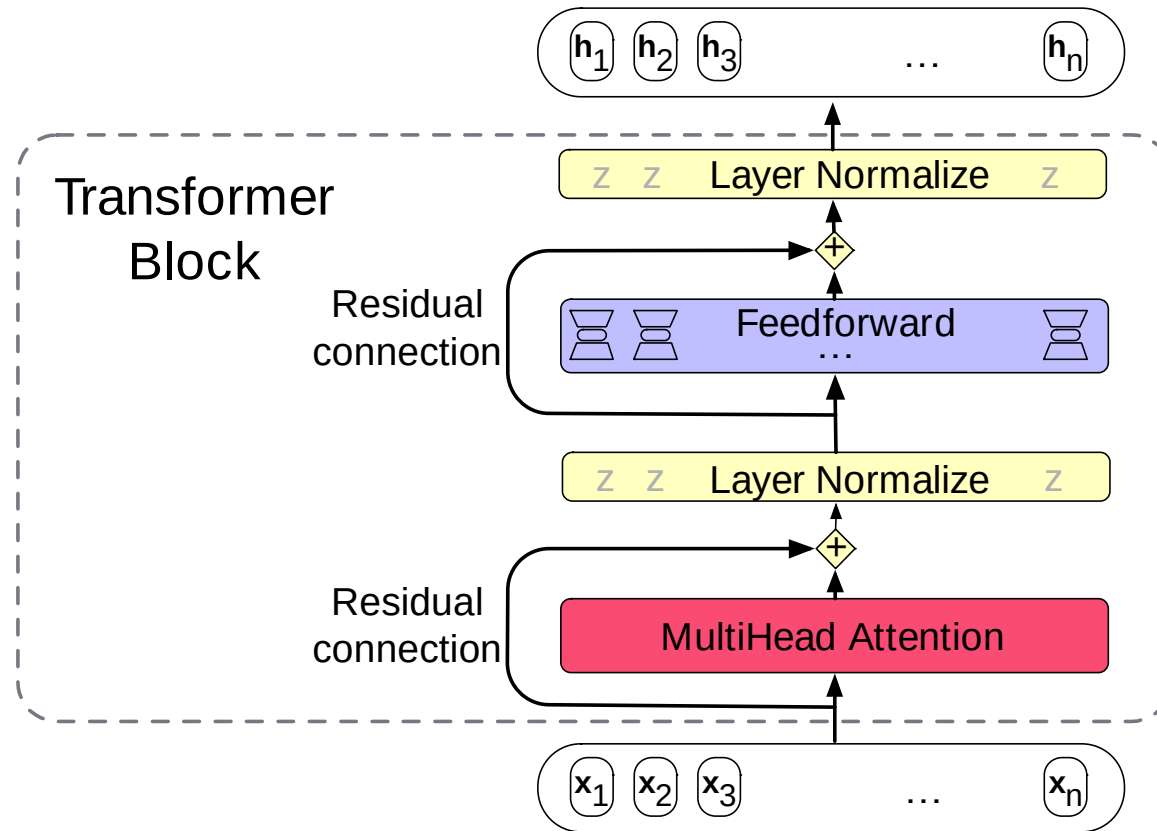


$$\mathbf{Q} = \mathbf{XW}_i^Q ; \mathbf{K} = \mathbf{XW}_i^K ; \mathbf{V} = \mathbf{XW}_i^V$$

$$\text{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\text{head}_1 \oplus \text{head}_2 \dots \oplus \text{head}_h) \mathbf{W}^O$$

Transformer Block



$$\mathbf{T}^1 = \text{SelfAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}_3 = \text{LayerNorm}(\mathbf{T}^2)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Both X and H are of $N \times d$

Transformer

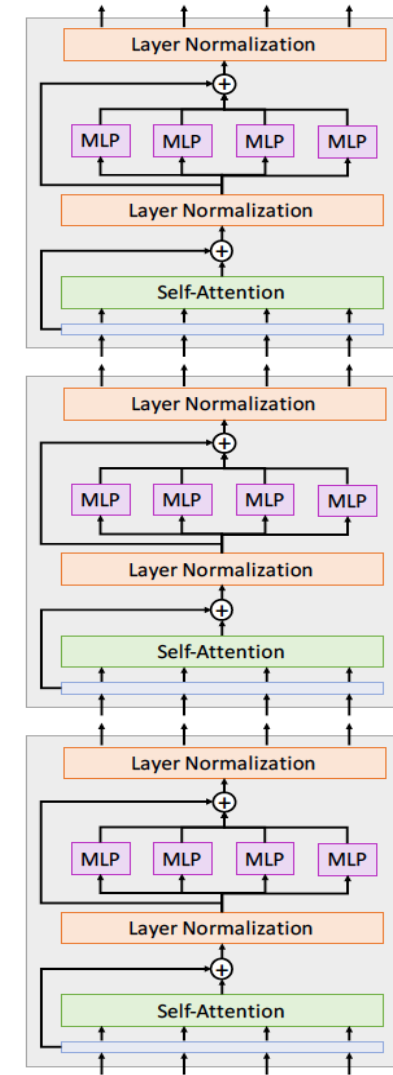
“ImageNet Moment for Natural Language Processing”

Pretraining:

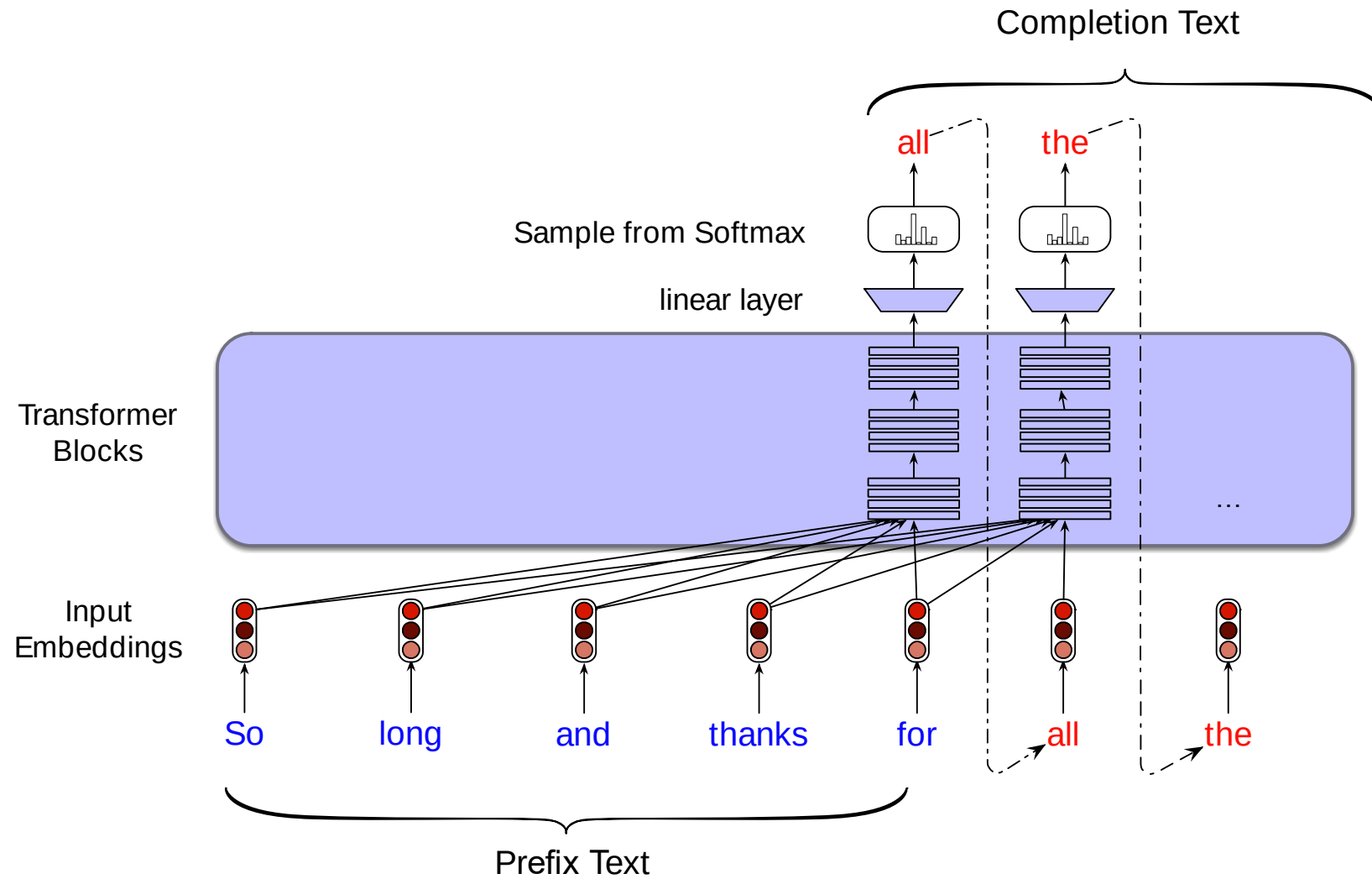
- Download a lot of text from the internet
- Train a giant Transformer model for language modeling

Finetuning:

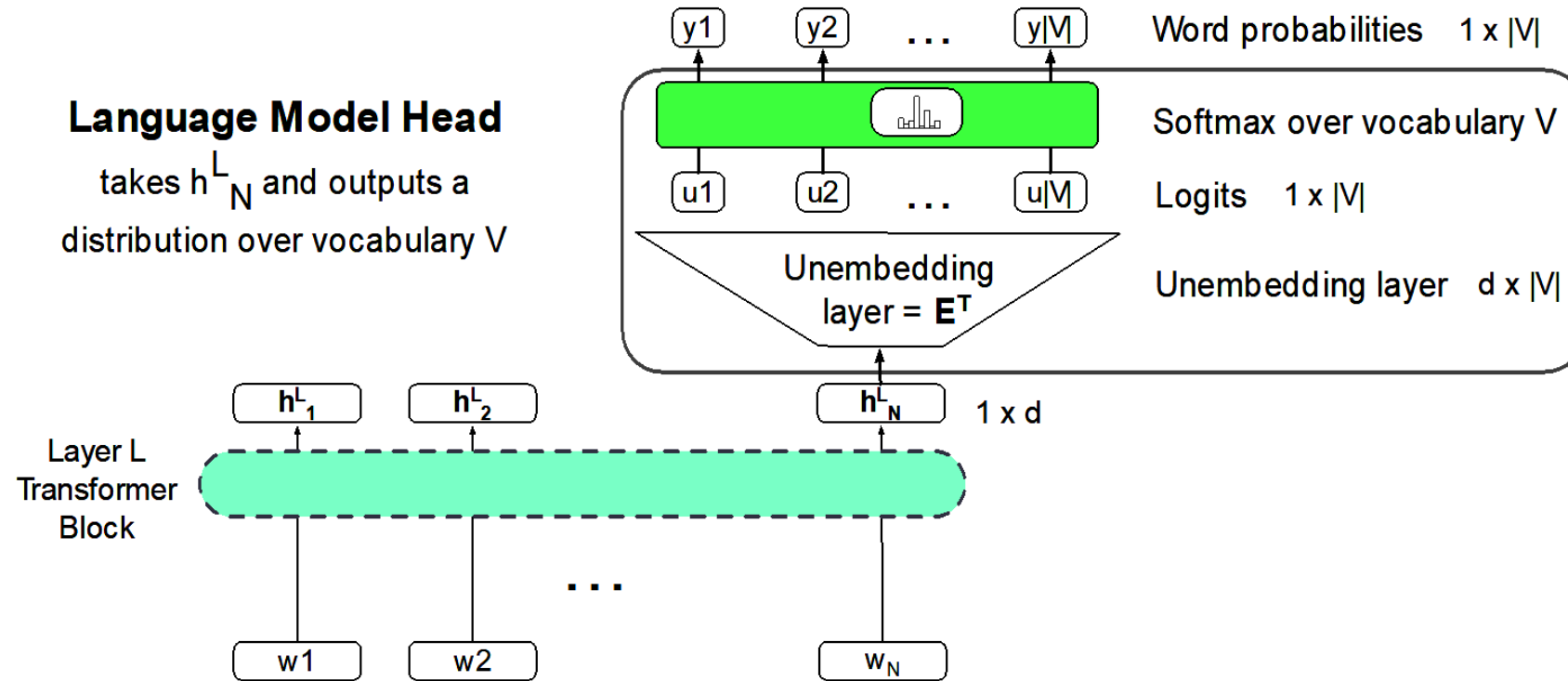
- Fine-tune the Transformer on your own NLP task



Sentence Completion

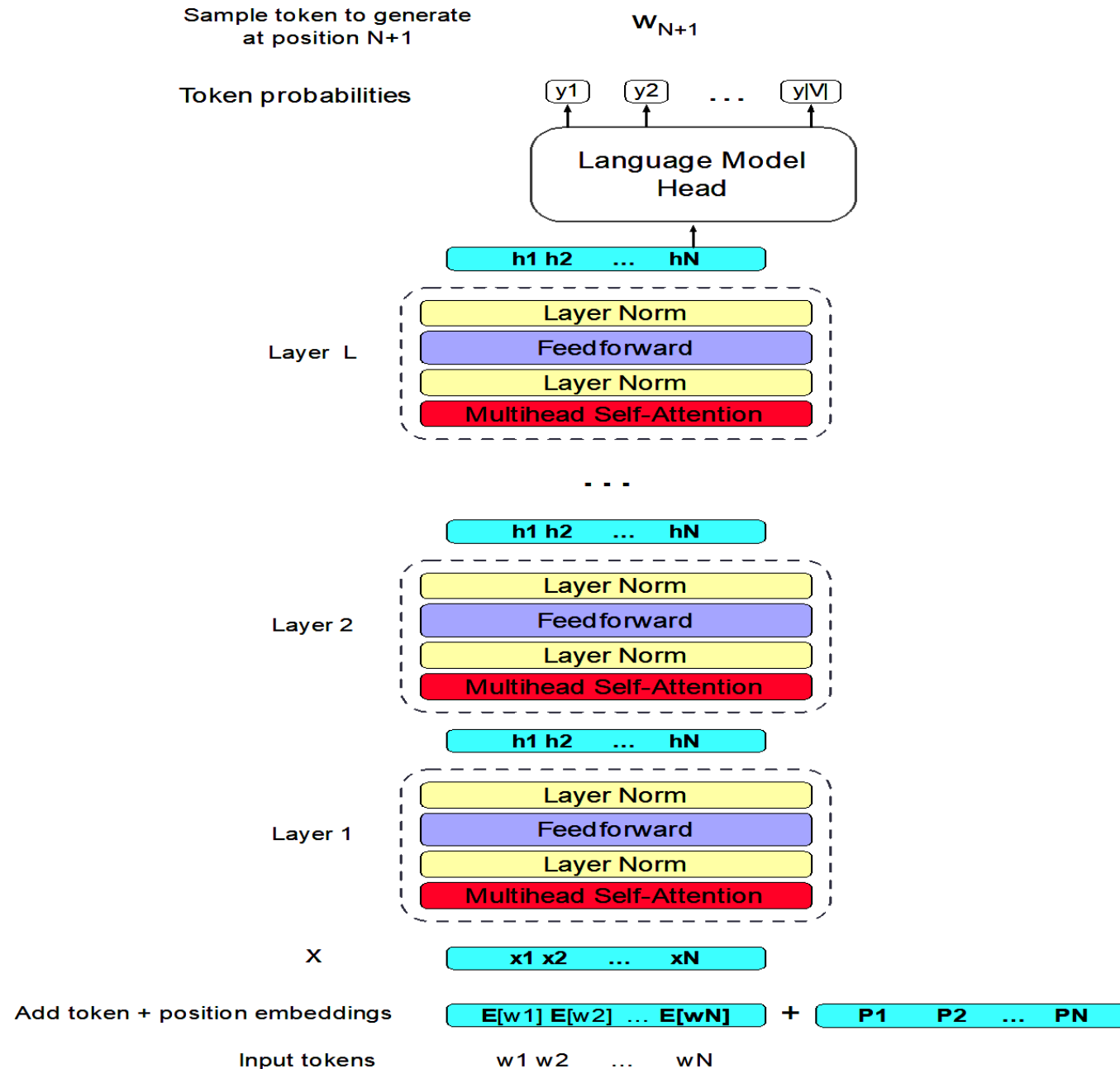


Language Modelling Head



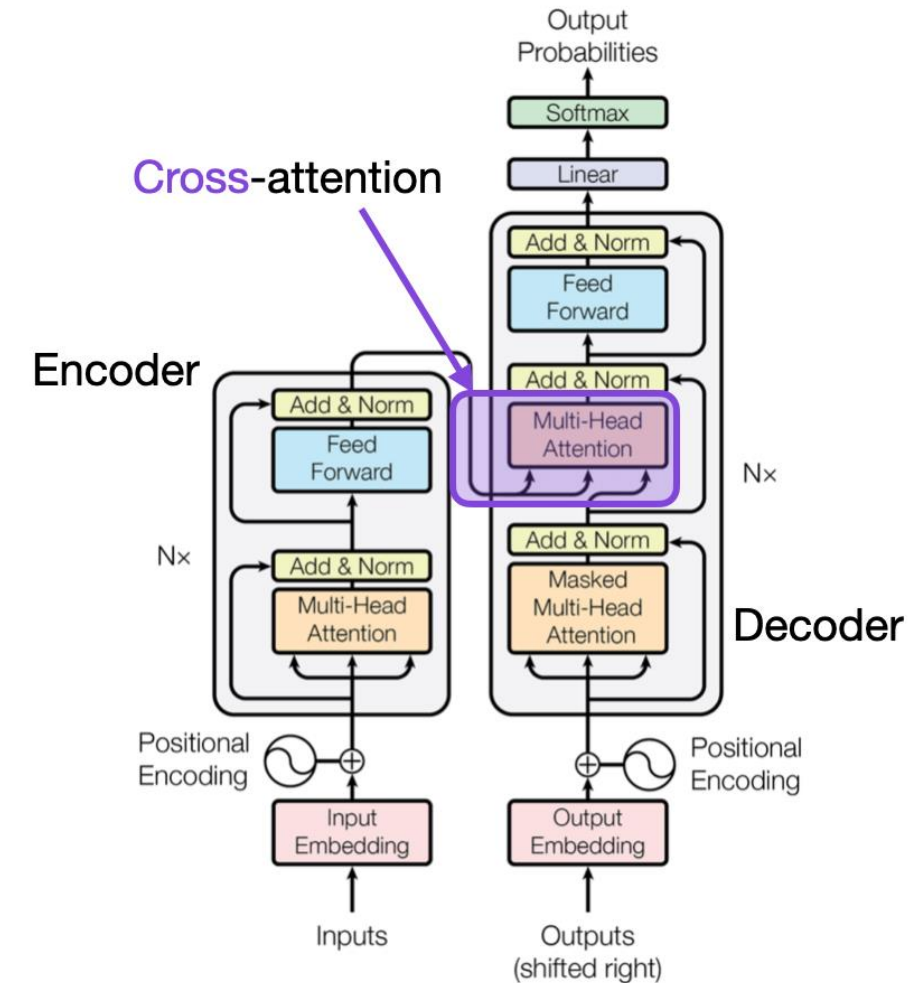
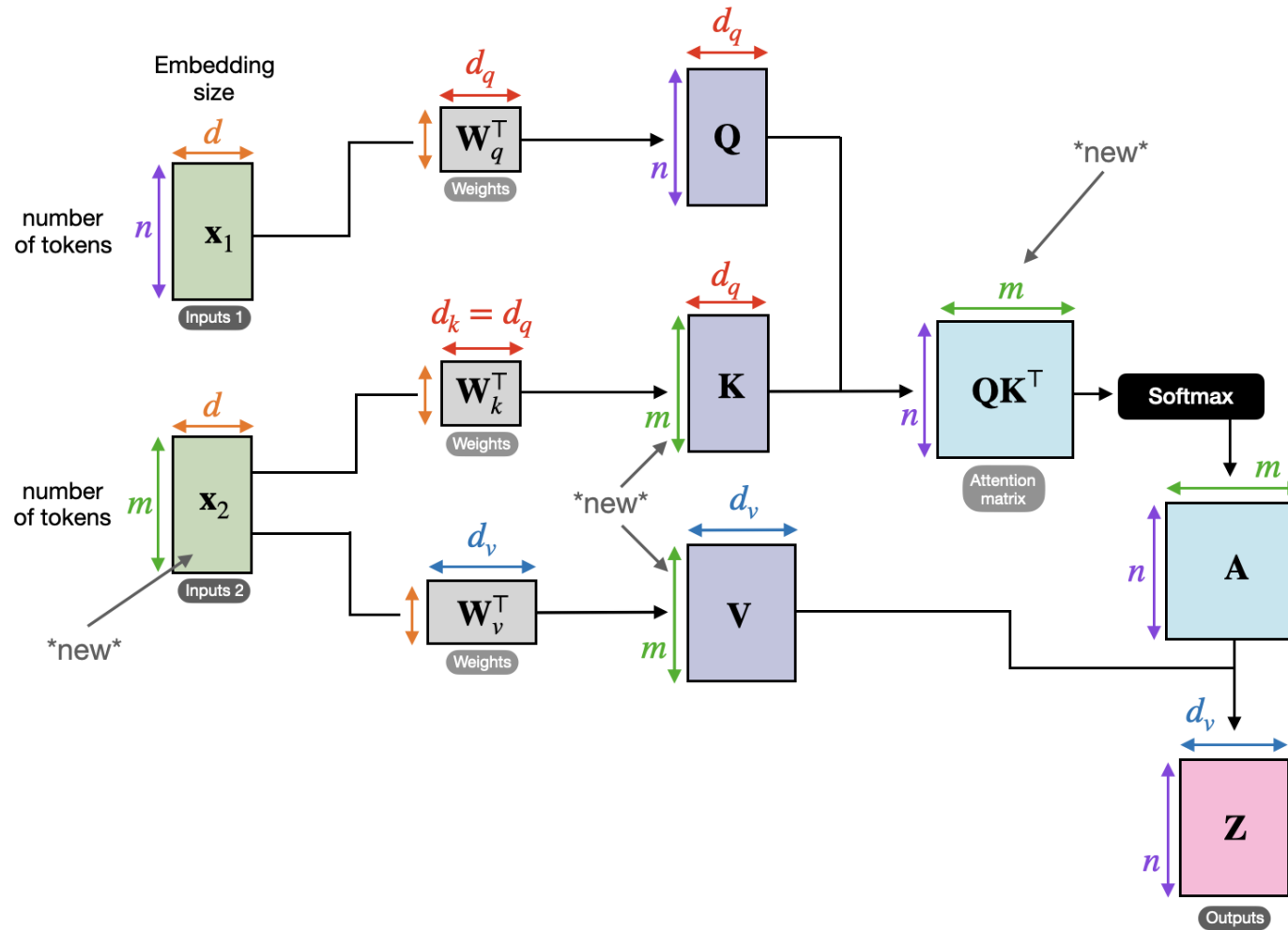
$$u = h_N^L E^T$$

$$y = \text{softmax}(u)$$



Eg. A Complete Decoder Model

Cross Attention

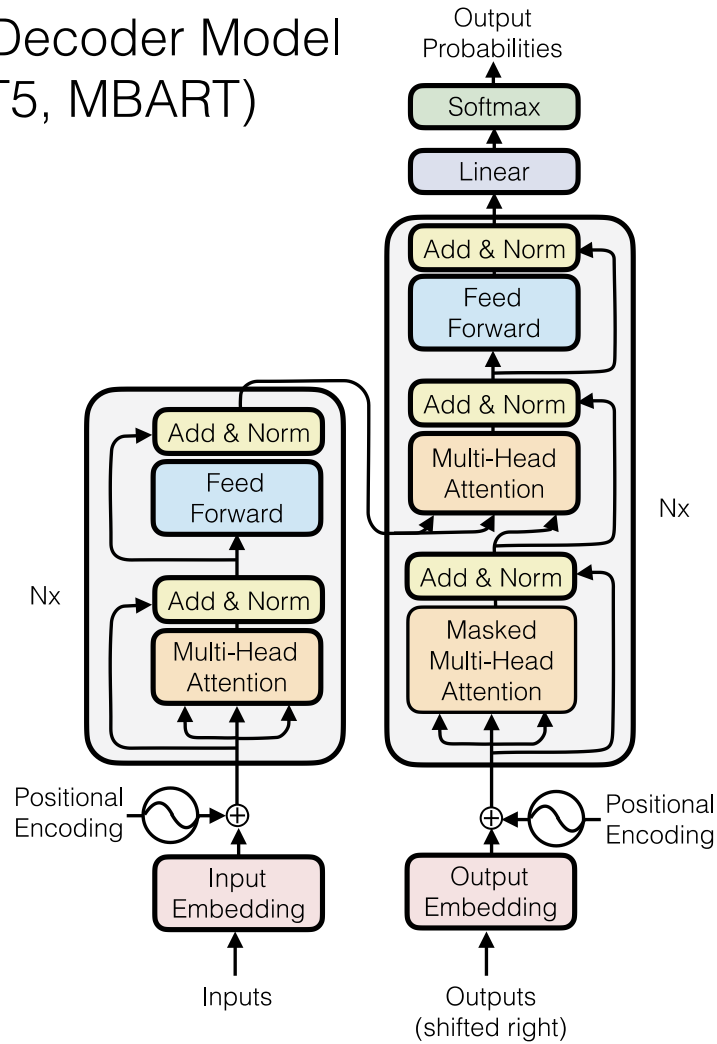


Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)

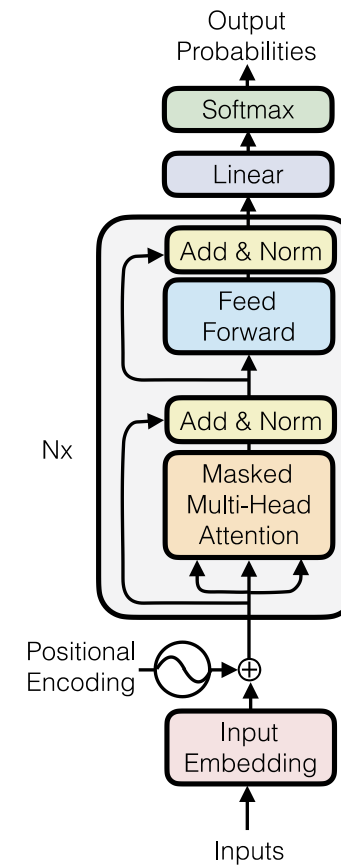
Example: Note that the queries usually come from the decoder, and the keys and values usually come from the encoder.

Transformers for NLP

Encoder-Decoder Model
(e.g. T5, MBART)



Decoder Only Model
(e.g. GPT, LLaMa)



Transformers for NLP

- Transformers¹, have revolutionized Natural Language Processing (NLP) tasks by introducing a novel architecture based on self-attention mechanisms
- Sample NLP tasks include:
 - Machine Translation
 - Language Understanding
 - Text Generation
 - Question Answering
 - Named Entity Recognition
- BERT [1], RoBERTa [2], DeBERTa [3], T5 [4], GPT v1-4 [10] represent major contributions in transformer-based models developed for these NLP tasks

NLP

- Self-supervised learning has driven the recent progress in the **Natural Language Processing** (NLP) field
 - Models like ELMO, BERT, RoBERTa, ALBERT, Turing NLG, GPT-3 have demonstrated immense potential for automated NLP
- Employing various pretext tasks for learning from raw text produced rich feature representations, useful for different downstream tasks
- **Pretext tasks** in NLP:
 - Predict the center word given a window of surrounding words
 - The word highlighted with green color needs to be predicted

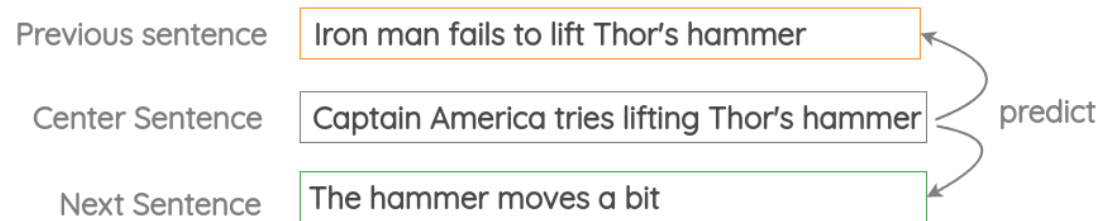


- Predict the surrounding words given the center word

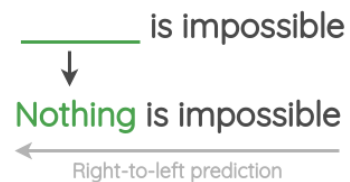
A quick brown fox jumps over the lazy dog

NLP

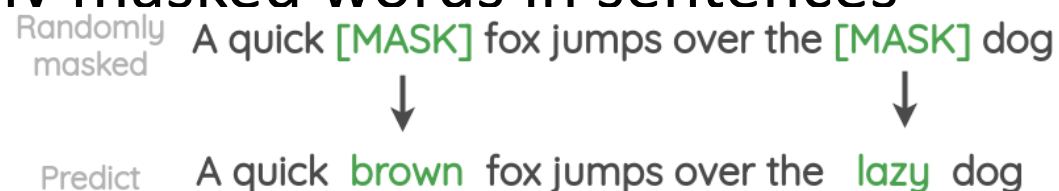
- **Pretext tasks** in NLP:
 - From three consecutive sentences, predict the previous and the next sentence, given the center sentence



- Predict the previous or the next word, given surrounding words



- Predict randomly masked words in sentences



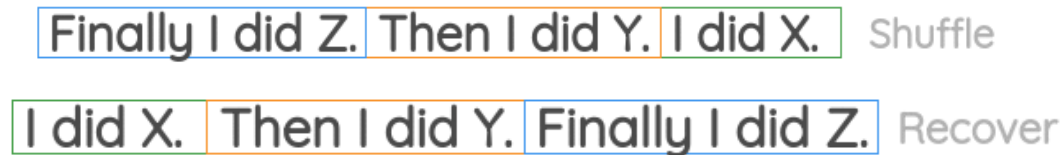
NLP

- **Pretext tasks** in NLP:

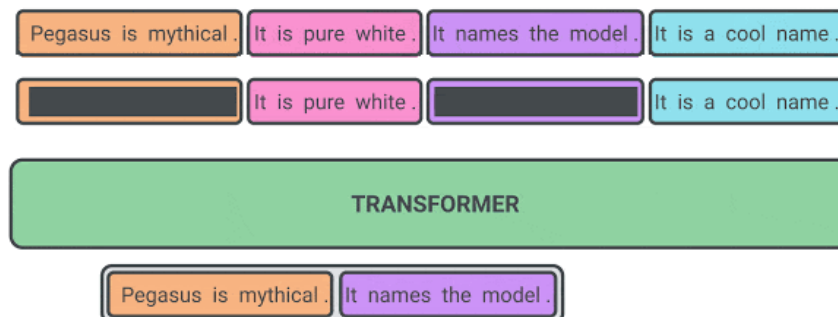
- Predict if the ordering of two sentences is correct

Sentence 1	Sentence 2	Next Sentence
I am going outside	I will be back in the evening	yes
I am going outside	You know nothing John Snow	no

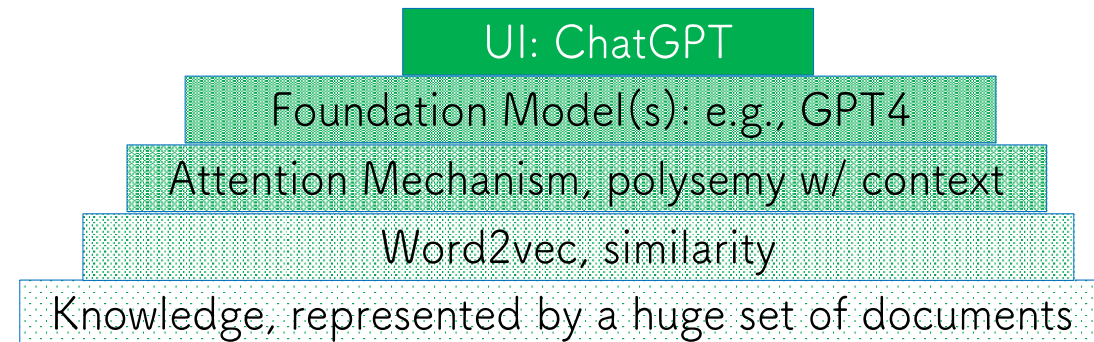
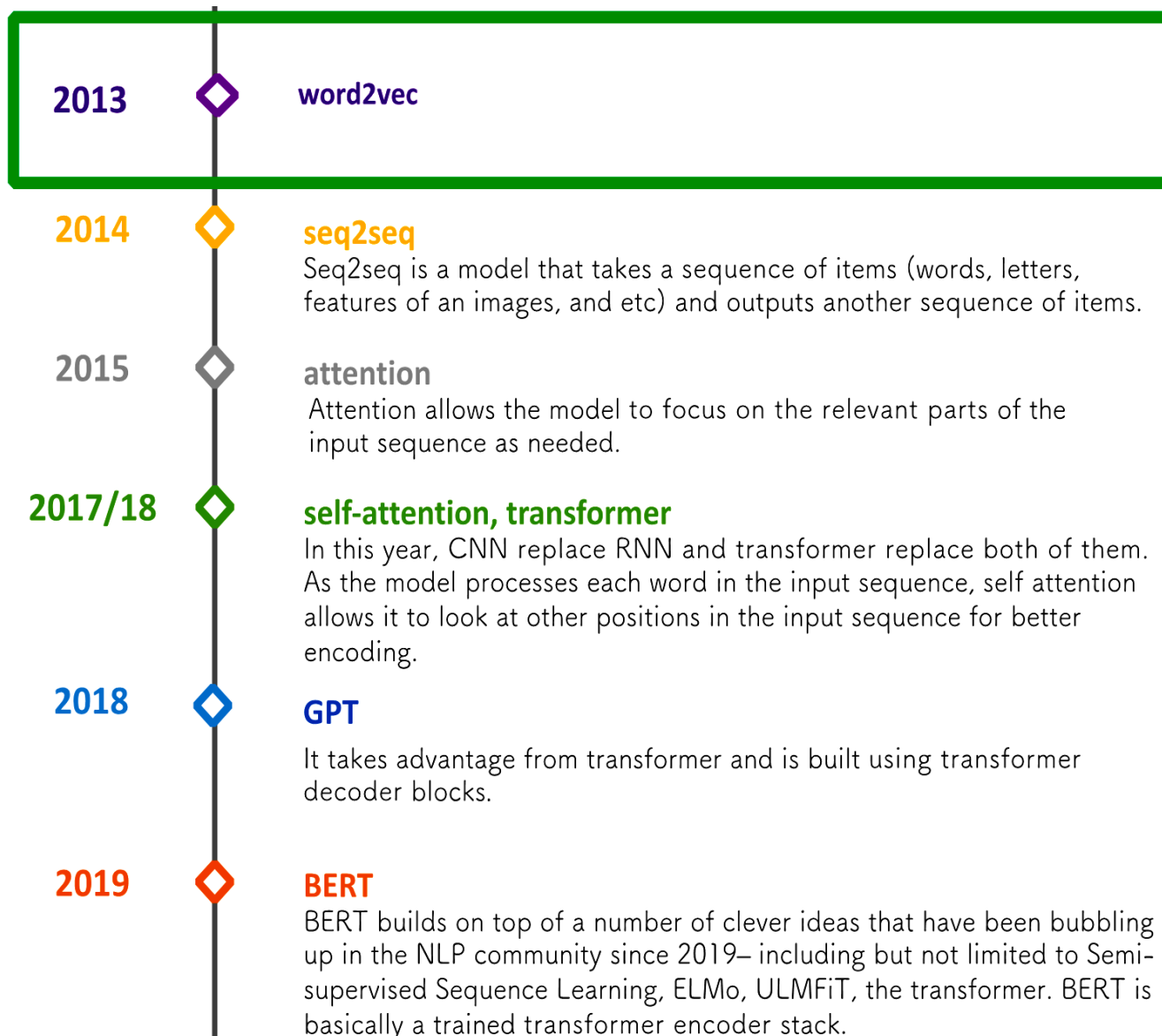
- Predict the order of words in a randomly shuffled sentence



- Predict masked



Improving Capability to Understand Language

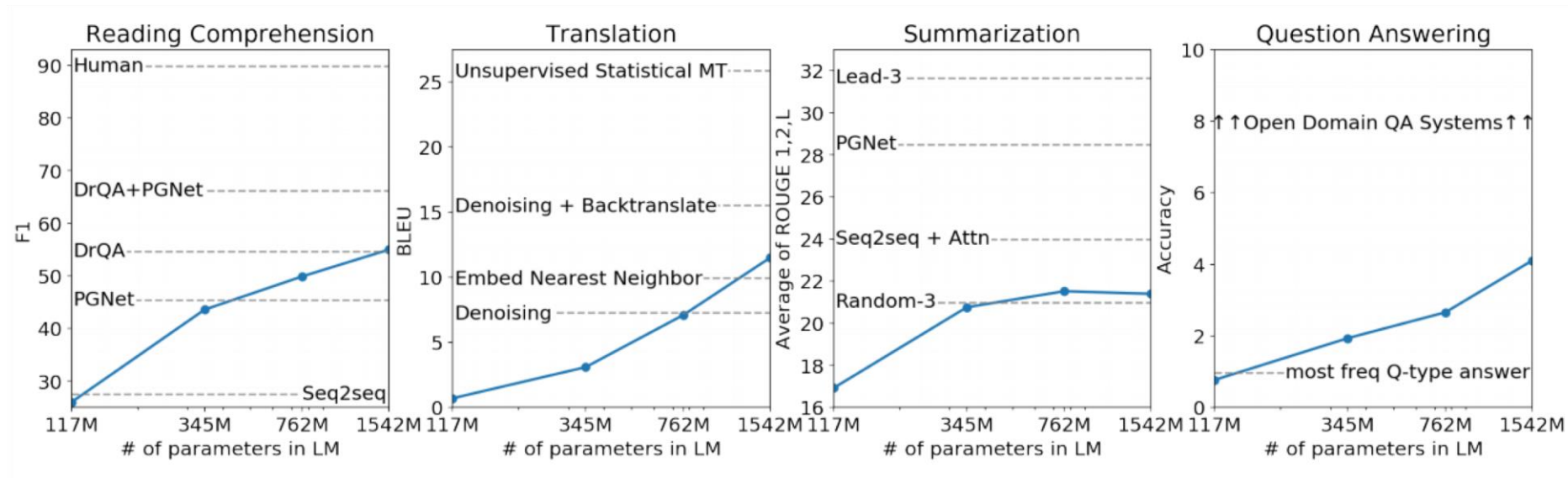


- Capability to complete sentences
- Capabilities to complete paragraphs
- Capability to answer questions
- Conditional generation of text/images
- Capability to harvest/use user feedbacks

Many Transformers

Transformers in NLP: GPT and GPT-2

- Radford et al., (2018) Language models are unsupervised multitask learners
 - Decoder transformer that predicts next word based on previous words by computing $P(x_t|x_{1..t-1})$
 - SOTA in “zero-shot” setting for 7/8 language tasks (where zero-shot means no task training, only unsupervised language modeling)



Transformers in NLP: BERT (Bidirectional Encoder Representations from Transformers)

- Devlin et al., (2019) BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
 - Decoder transformer that predicts a missing word based on surrounding words by computing $P(x_t | x_{1..t-1, t+1..T})$
 - Mask missing word with masked multi-head attention
 - Improved state of the art on 11 tasks

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

GPT-3

- **GPT-3** stands for **Generative Pre-trained Transformer**
 - It was created by [OpenAI](#), and introduced in May 2020
- **Transformers** are currently the most common model architecture for NLP tasks
 - They employ attention blocks for discovering correlation in text
- GPT-3 generates text based on initial input prompt from the end-user
 - It is trained using next word prediction on huge amount of raw text from the internet
 - The quality of text generated is often undistinguishable from human-written text
 - GPT-3 can also be used for other tasks, such as answering questions, summarizing text, automated code generation, and many others
- It is a NN model at the present, having 175 billion parameters
 - The cost for training GPT-3 reportedly is \$ 12 million!!

Examples

- “Original Transformer” (2017)
 - MT; 512 dim; 8 heads; 2X6 blocks
 - Trained on 3.5 days on 8 GPUs
- BERT (2018)
 - General-Language model; Non-causal; 350M parameters
 - 1024 dim; 16 heads; 24 blocks; $l = 512$;
 - Trained on 4 days 16 TPUs

Examples

- GPT2 (2019)
 - Causal; 1.5 B parameters
 - D = 768; 14 heads; 48 blocks;
 - Trained on 7 days; 256 TPUs
- GPT3(2020)
 - Causal; 175B parameters
 - D = 12288; 96 heads; 96 blocks; seq 2048
 - Trained on 10K GPUs; 12 days; USD 4,600,000

Comments: Empirical Performances

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

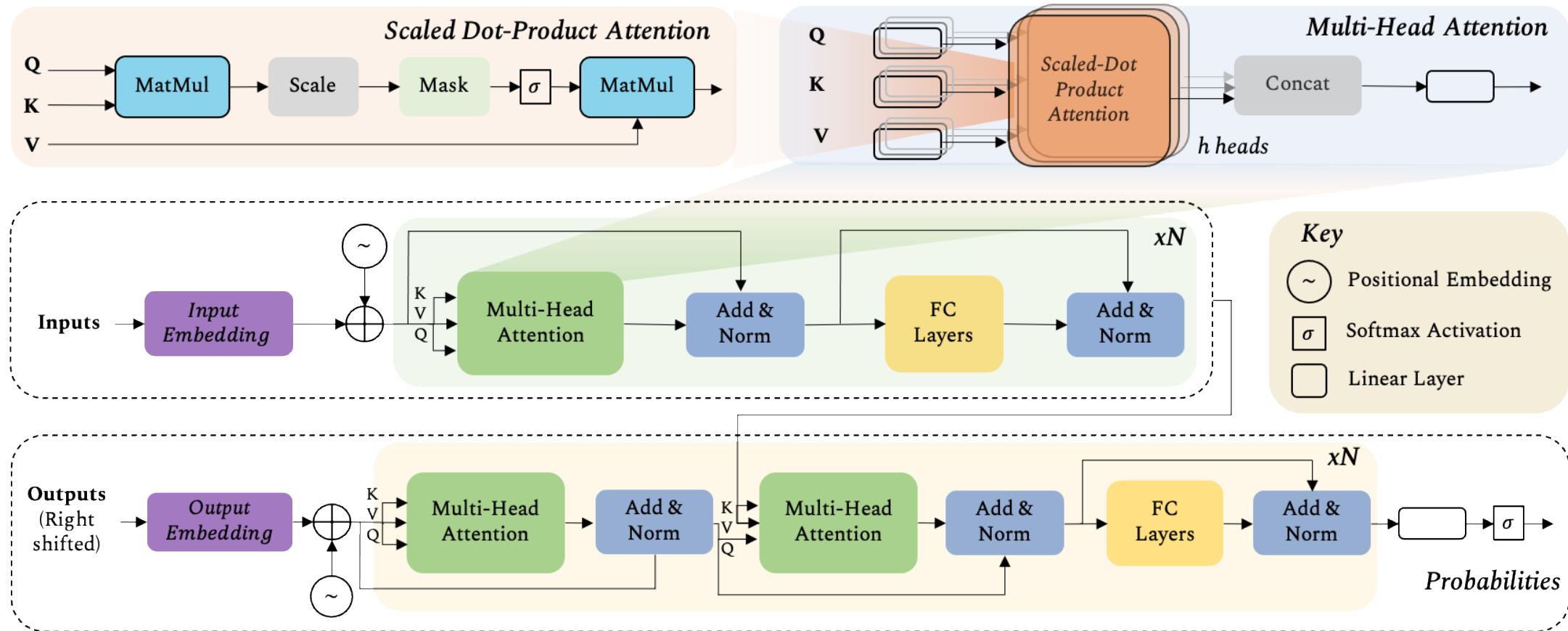
Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.0	$2.3 \cdot 10^{19}$	

Comments: Complexity

- Attention reduces sequential operations and maximum path length, which facilitates long range dependencies

Layer-type	Complexity per Layer	Sequential Operations
Recurrent	$O(n \cdot d^2)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$
Transformer	$O(n^2 \cdot d)$	$O(1)$
Sparse Transformer	$O(n\sqrt{n})$	$O(1)$
Reformer	$O(n \log(n))$	$O(\log(n))$
Linformer	$O(n)$	$O(1)$
Linear Transformer	$O(n)$	$O(1)$

Summary



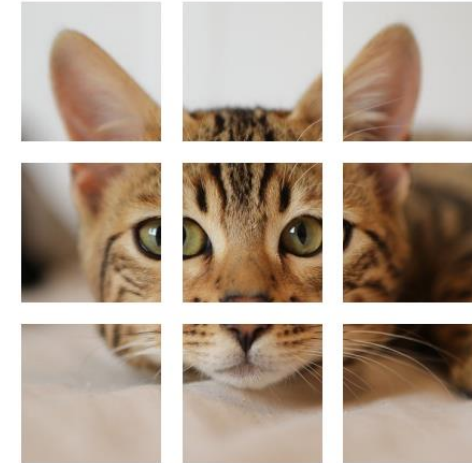
Blank

Vision Transformers

Image to Tokens



224 X 224

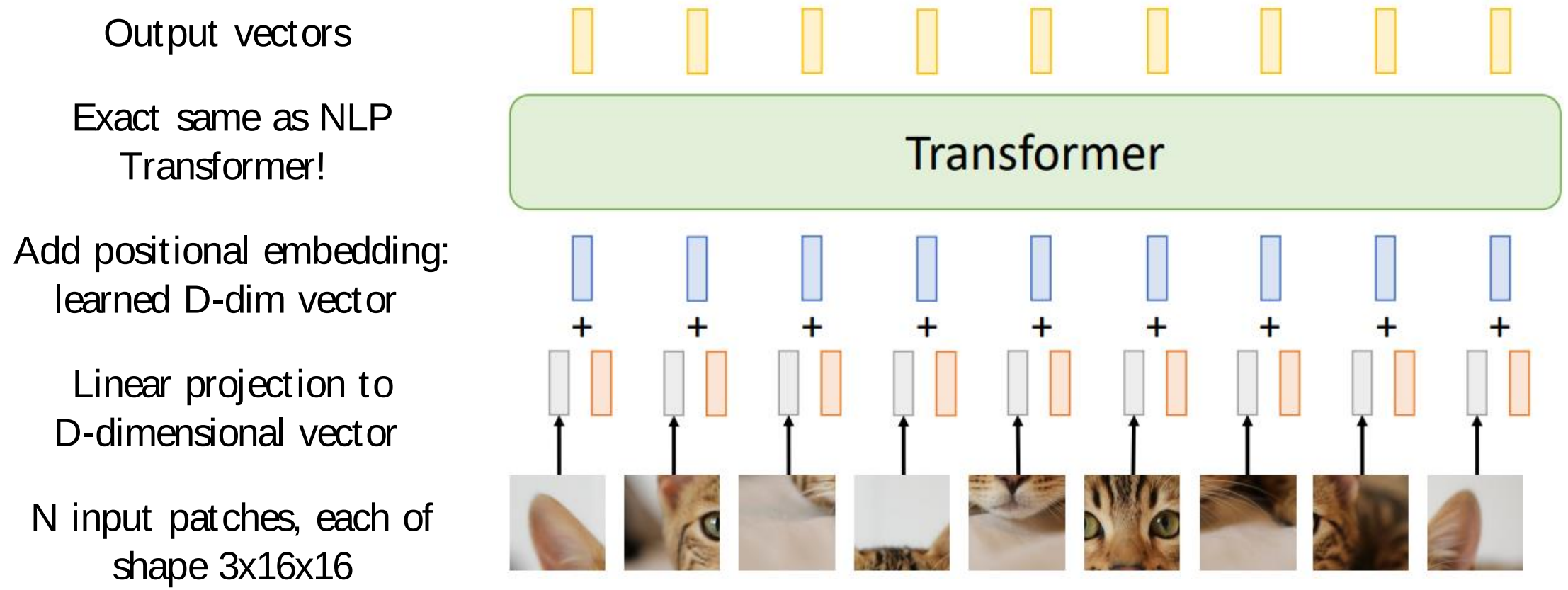


Divide the image into 196 (14 → 14)
small images of size 16 → 16

N input patches, each of
shape 3x16x16



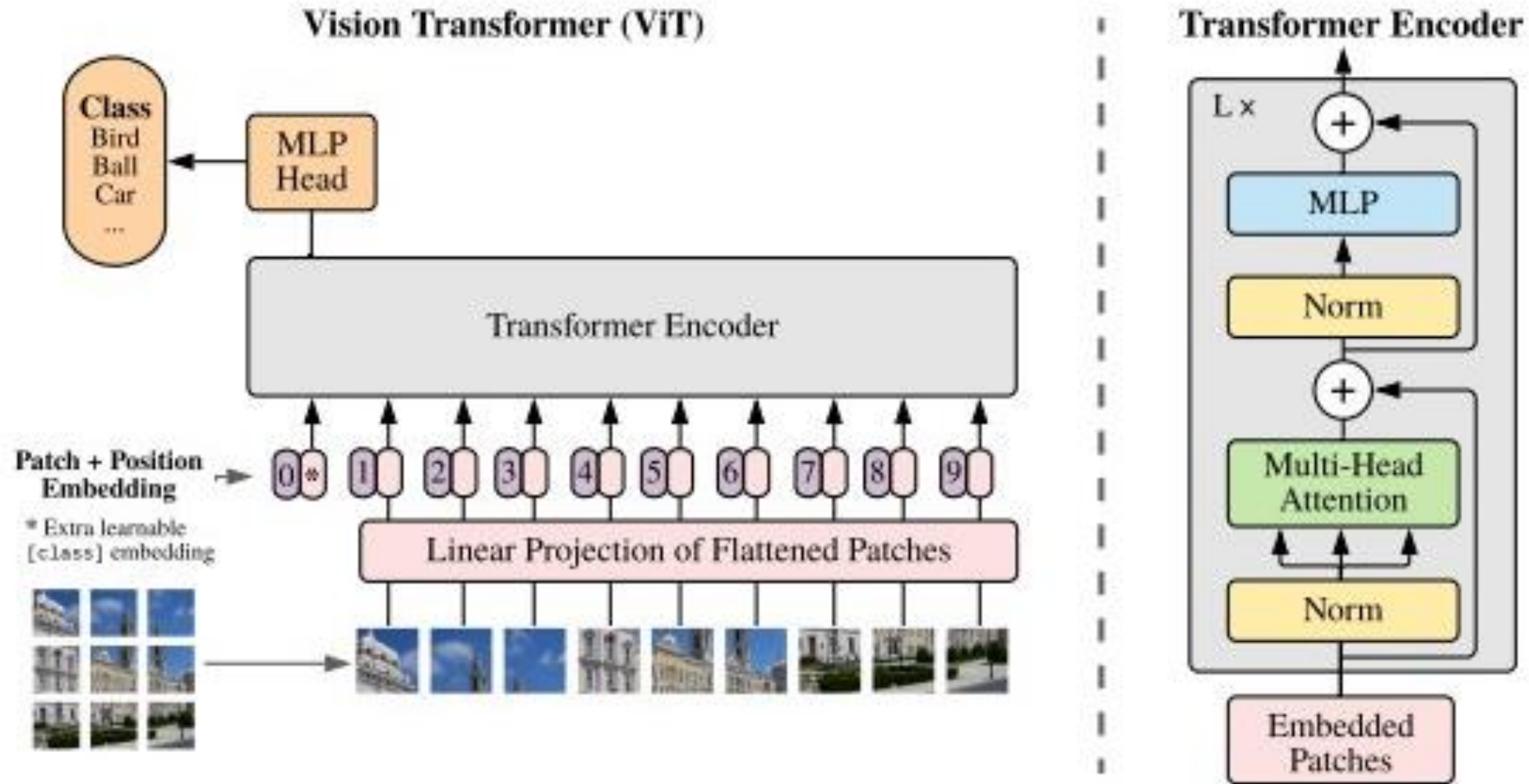
Vision Transformer



Basic Steps

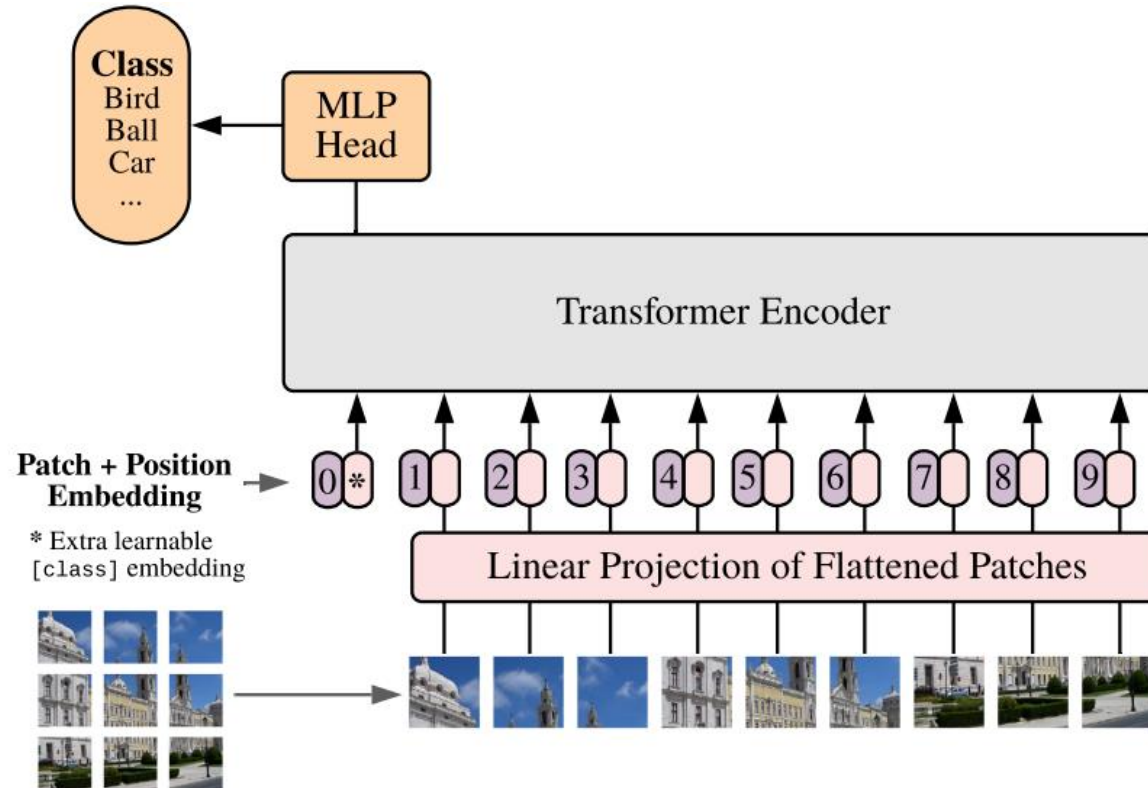
1. Split an image into patches
2. Flatten the patches
3. Produce lower-dimensional linear embeddings from the flattened patches
4. Add positional embeddings
5. Feed the sequence as an input to a standard transformer encoder
6. Pretrain the model with image labels (fully supervised on a huge dataset)
7. Finetune on the downstream dataset for image classification

Vision Transformer



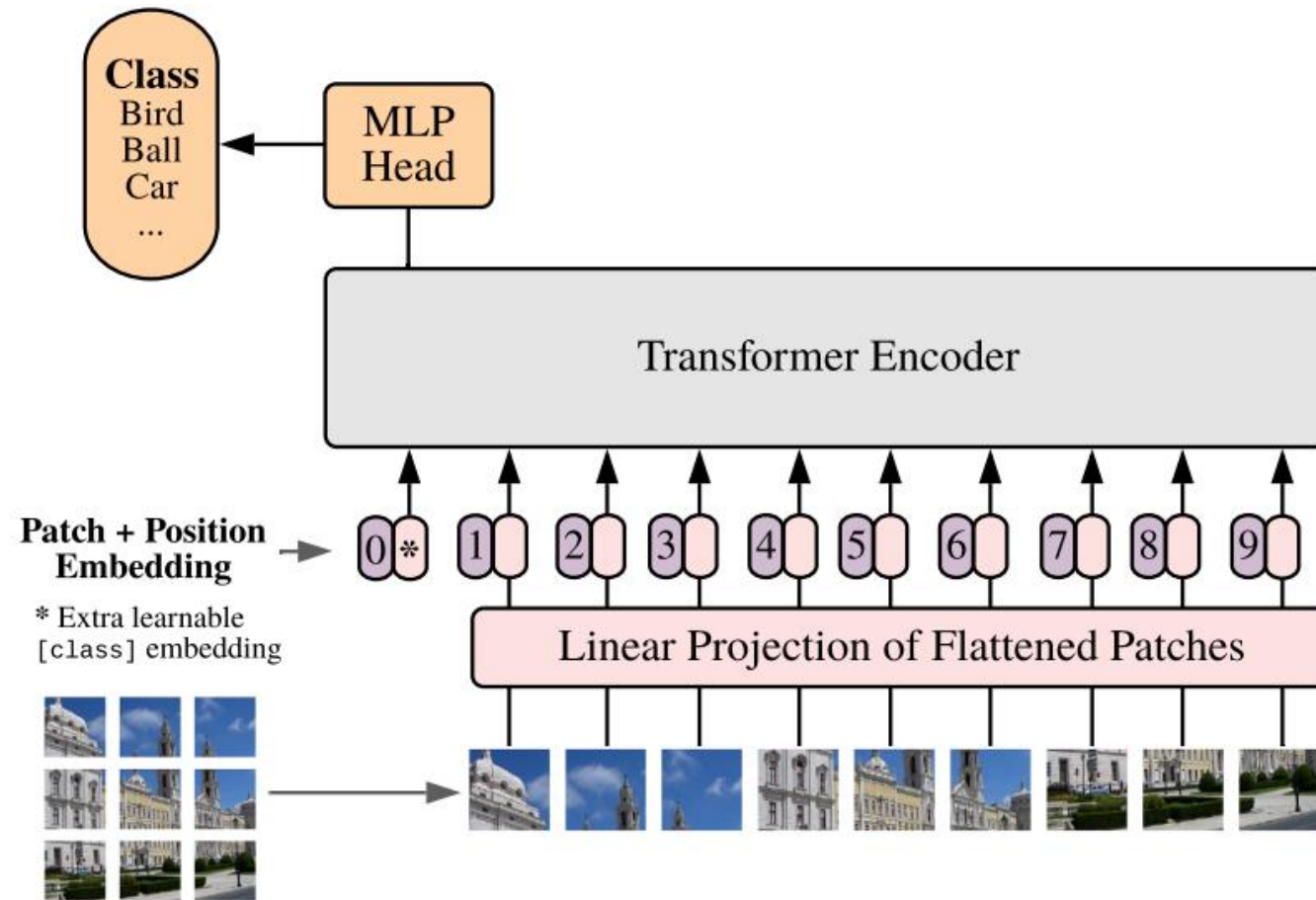
Architecture resembles the one used in language; 'An image is worth 16X16 words, 2020

Patches; Not Pixels



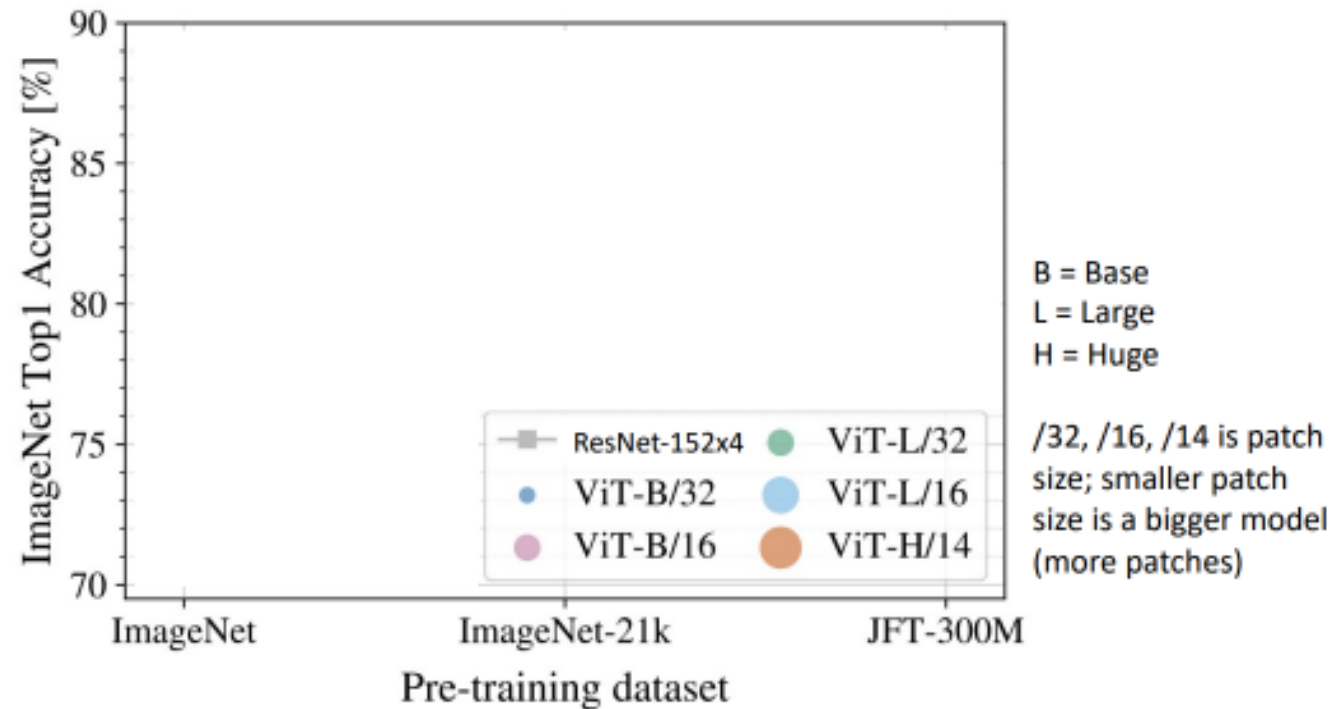
- Using a pixel as a token, for an image of size $3 \times 224 \times 224$, we had $(3 \times 224 \times 224)^2 = 22$ billion parameters for self-attention calculation. Now?
- $(14 \times 14)^2 = 38,416$ parameters (approx 150 KB) per transformer layer!

Convolutions ?



- Do we have a vision architecture without convolution?!
- No!** The linear projection from patches is a convolution (with stride being patch size)
- Also, MLPs in Transformer are stacks of 1×1 convolution

ViT Vs ResNet

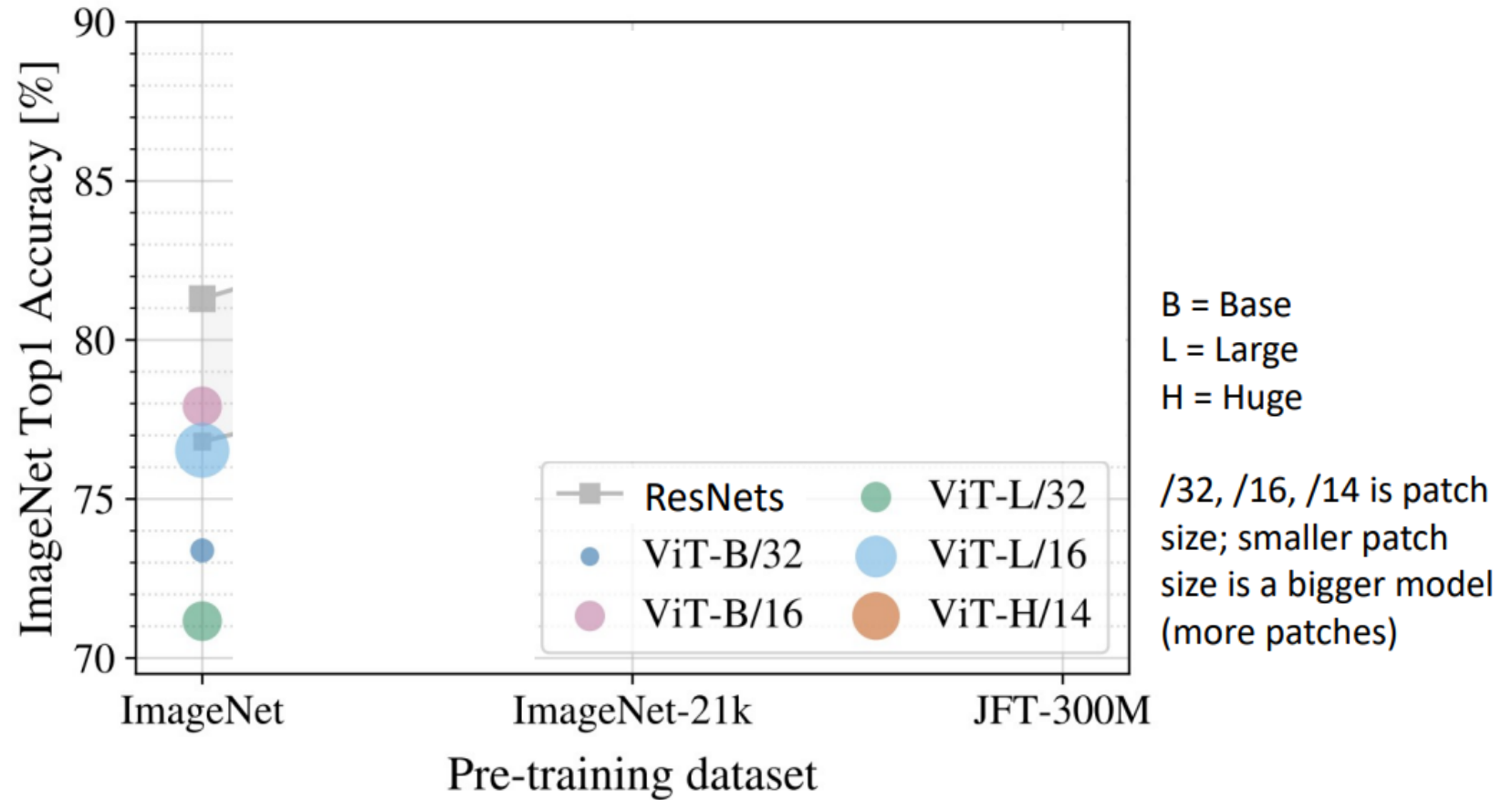


Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

ViT vs ResNet

Recall: ImageNet dataset has 1k categories, 1.2M images

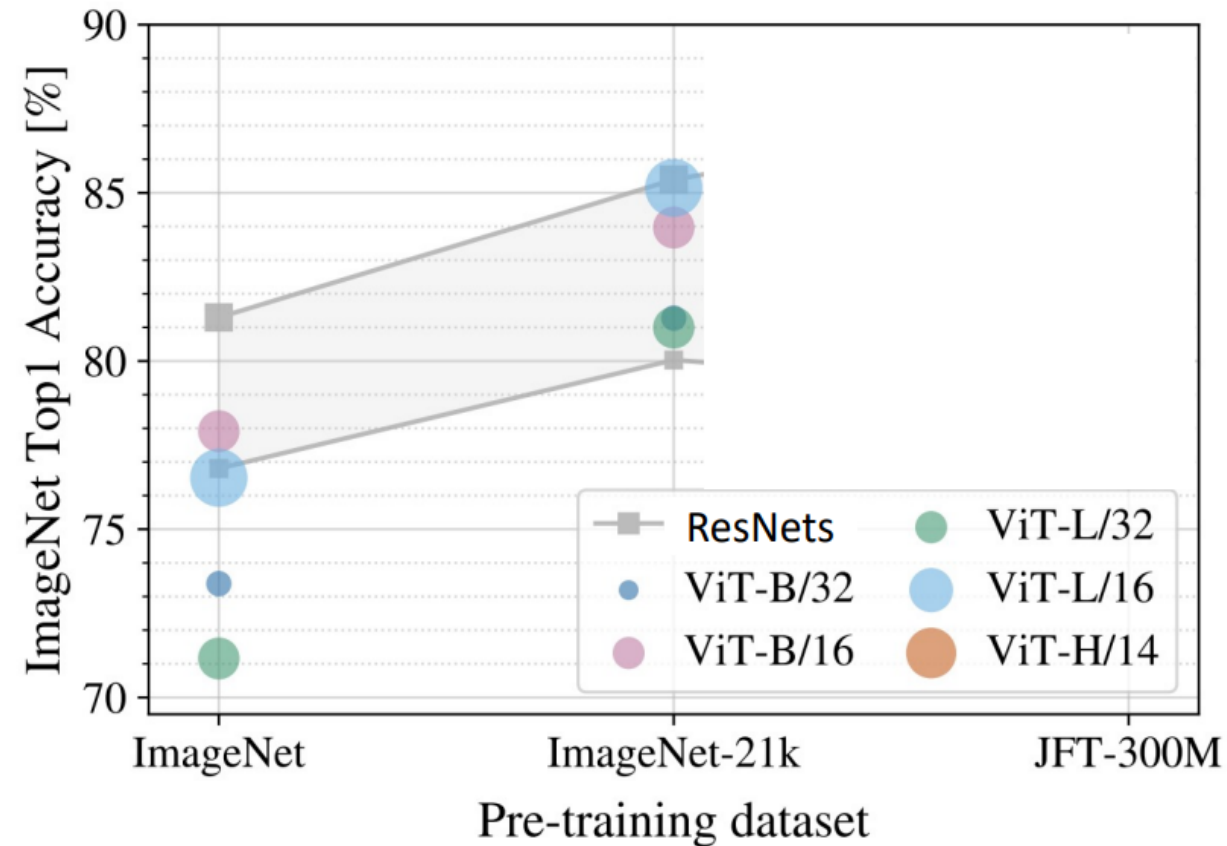
When trained on ImageNet, ViT models perform worse than ResNets



Vision Transformer vs Resnet

ImageNet-21k has 14M images with 21k categories

If you pretrain on ImageNet-21k and fine-tune on ImageNet, ViT does better: big ViTs match big ResNets



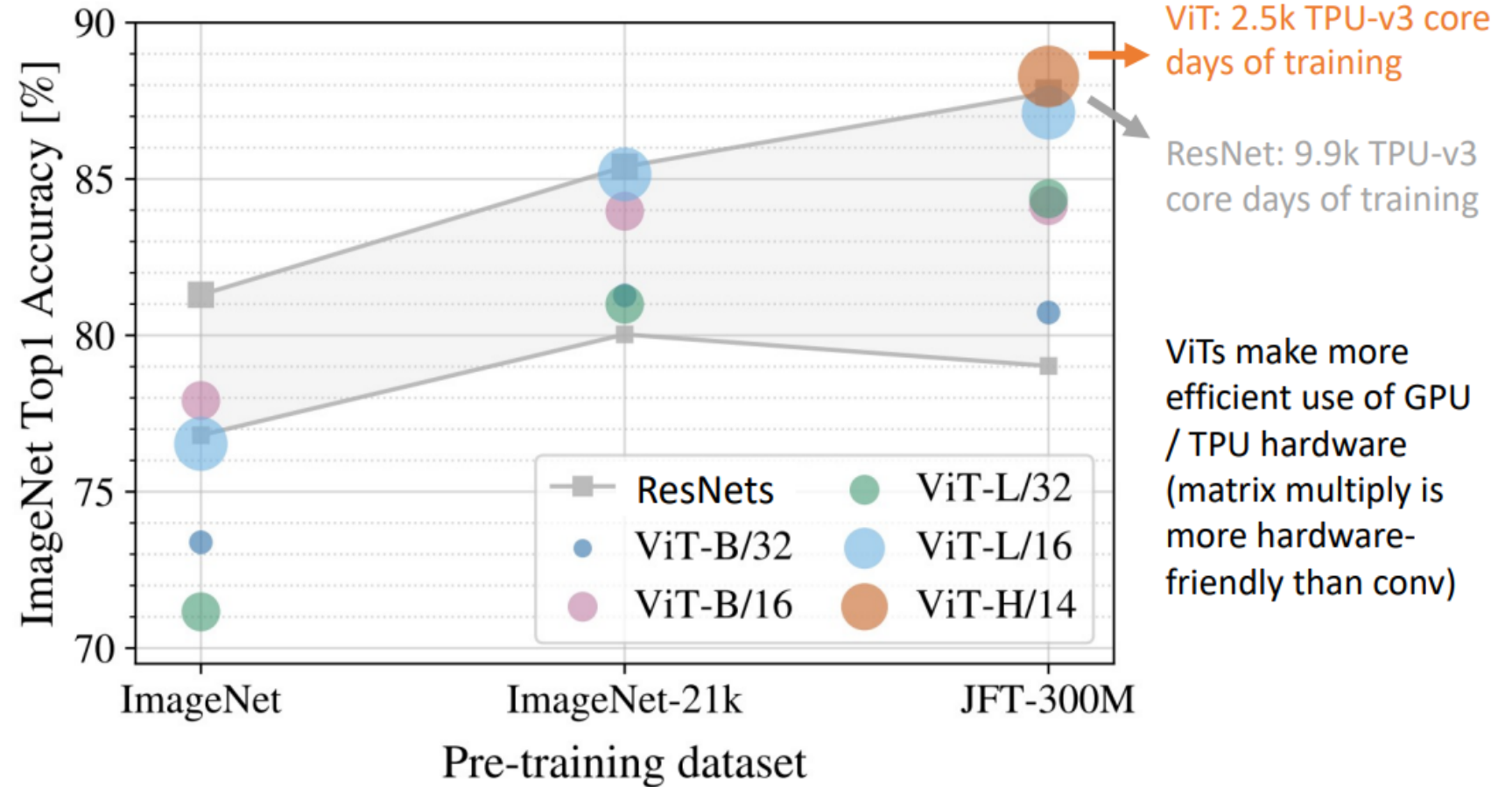
B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

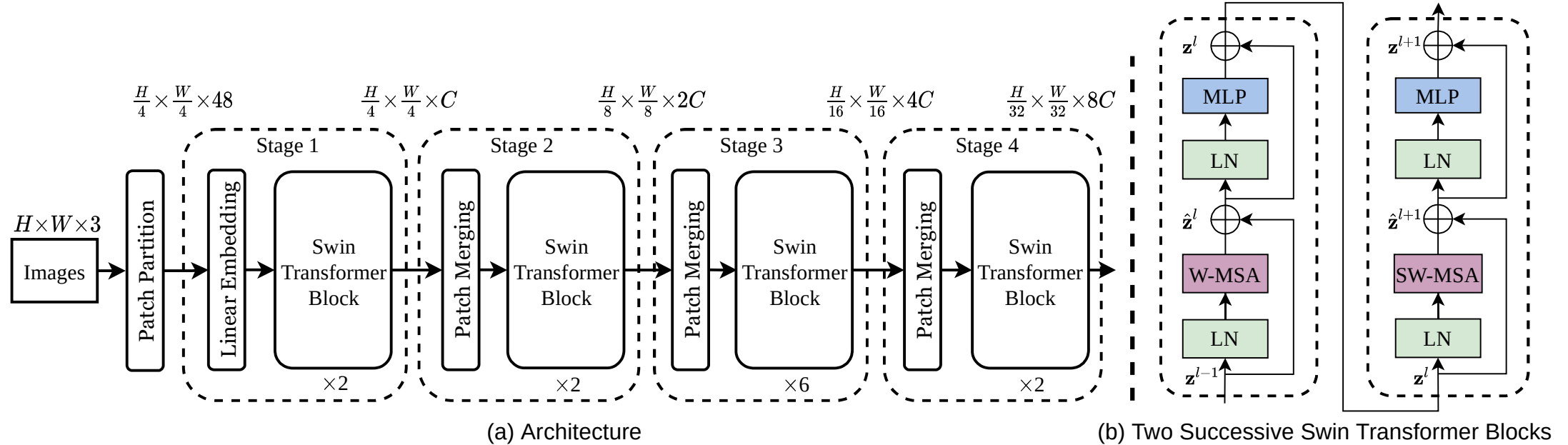
Vision Transformer vs ResNet

JFT-300M is an internal Google dataset with 300M labeled images

If you pretrain on JFT and finetune on ImageNet, large ViTs outperform large ResNets



Vision Transformers: Swin Transformer



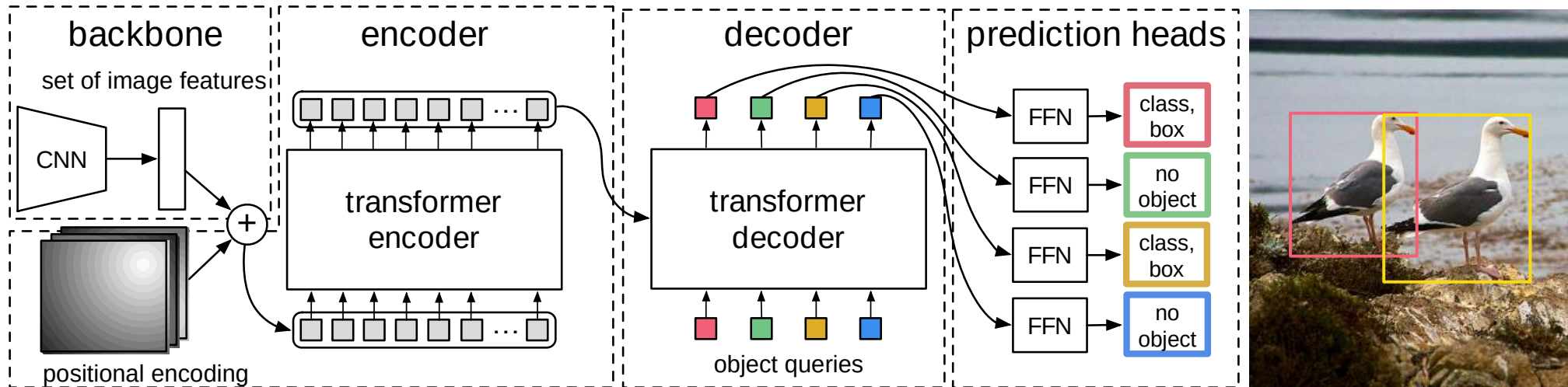
- High variation in scales of objects (vs words in language)
- High resolution of pixels
- Hierarchical transformer with Swin = shifter windows
- Act as general purpose backbone

ImageNet Classification: Many Models

Method	#Param (M)	GFLOPs	Top-1 Acc (%)	Method	#Param (M)	GFLOPs	Top-1 Acc (%)
ResNet18 [67]★	11.7	1.8	69.8	ResNet101 [67] ★	44.7	7.9	77.4
EfficientNet-B3 [87]★	12.0	1.8	81.6	ResNeXt101-32x4d [244]★	44.2	8.0	78.8
DeiT-T [12]	5.7	1.3	72.2	RegNetY-8G [86]★	39.0	8.0	81.7
T2T-ViT _t -7 [35]	5.0	1.3	71.7	EfficientNet-B5 [87] ★	30.0	9.9	83.6
LocalViT-T [107]	5.9	1.3	74.8	CvT-21 [96]	32.0	7.1	82.5
CrossViT-T [104]	6.9	1.6	73.4	CaiT-S-24 [243]	32.2	9.4	82.7
PVTv1-T [93]	13.2	1.9	75.1	T2T-ViT _t -19 [35]	39.0	9.8	81.4
ResT-Lite [110]	10.5	1.4	77.2	PVTv1-M [93]	44.2	6.7	81.2
CaiT-XXX-24 [243]	12.0	2.5	77.6	PVTv2-B3 [97]	45.2	6.9	83.2
PVTv2-B1 [97]	13.1	2.1	78.7	NesT-S [111]	38.0	10.4	83.3
Lv-ViT-T [89]	8.5	–	79.1				
RegionViT-T [100]	13.8	2.4	80.4	ResNet152 [67] ★	60.2	11.6	78.3
ResNet50 [67]★	25.6	4.1	76.1	CaiT-S-36 [243]	48.0	13.9	83.3
ResNeXt50-32x4d [244]★	25.0	4.3	77.6	T2T-ViT _t -24 [35]	64.0	15.0	82.2
RegNetY-4G [86]★	21.0	4.0	80.0	PVTv1-L [93]	61.4	9.8	81.7
EfficientNet-B4 [87]★	19.0	4.2	82.9	TNT-B [88]	66.0	14.1	82.8
DeiT-S [12]	22.1	4.6	79.9	Swin-S [36]	50.0	8.7	83.0
PVTv1-S [93]	24.5	3.8	79.8	Twins-SVT-B [37]	56.0	8.3	83.2
LocalViT-S [107]	22.4	4.6	80.8	RegionViT-B [100]	72.7	13.0	83.3
CrossViT-S [104]	26.7	5.6	81.0	PVTv2-B4 [97]	62.6	10.1	83.6
TNT-S [88]	23.8	5.2	81.3				
Swin-T [36]	29.0	4.5	81.3	ResNeXt101-64x4d [244] ★	83.5	15.6	79.6
NesT-T [111]	17.0	5.8	81.5	RegNetY-16G [86] ★	84.0	16.0	82.9
T2T-ViT _t -14 [35]	21.5	5.2	81.5	EfficientNet-B6 [87] ★	43.0	19.0	84.0
CvT-13 [96]	20.0	4.5	81.6	NesT-B [111]	68.0	17.9	83.8
ResT-B [110]	30.3	4.3	81.6	ViT-B/16 [11]	86.6	17.6	79.8
Twins-SVT-S [37]	24.0	2.8	81.7	DeiT-B/16 [12]	86.6	17.6	81.8
PVTv2-B2-Li [97]	22.6	3.9	82.1	Swin-B [36]	88.0	15.4	83.3
RegionViT-S [100]	30.6	5.6	82.5	Twins-SVT-L [37]	99.2	14.8	83.7
Lv-ViT-S [89]	26.0	6.6	83.3	PVTv2-B5 [97]	82.0	11.8	83.8
				Lv-ViT-M [89]	56.0	16.0	84.1

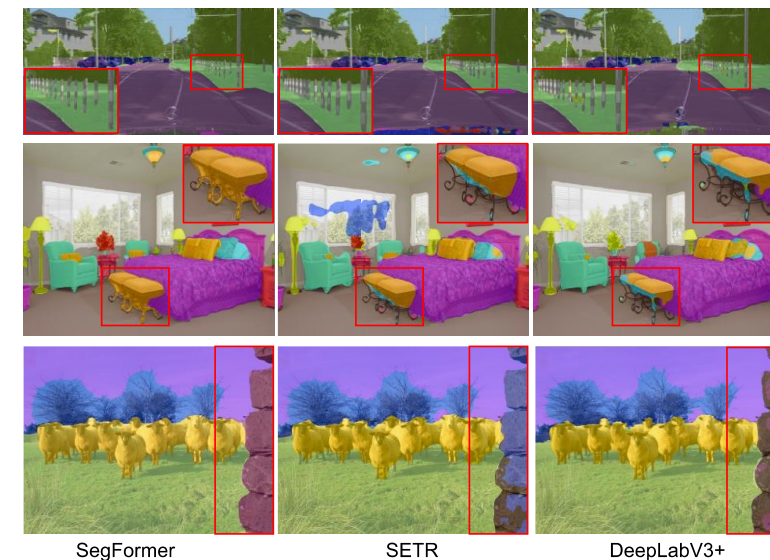
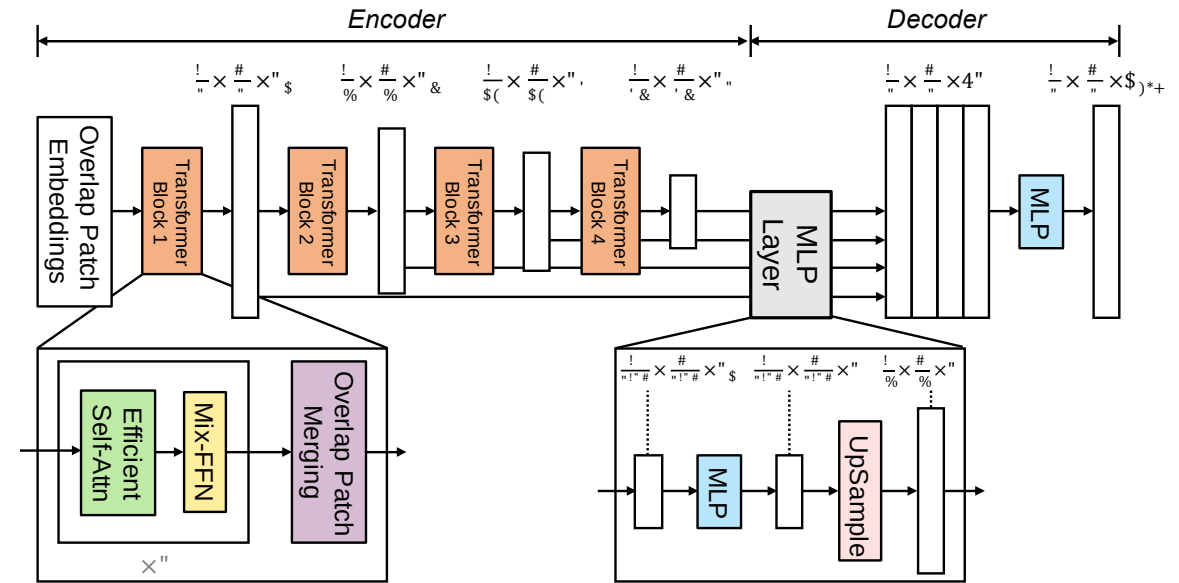
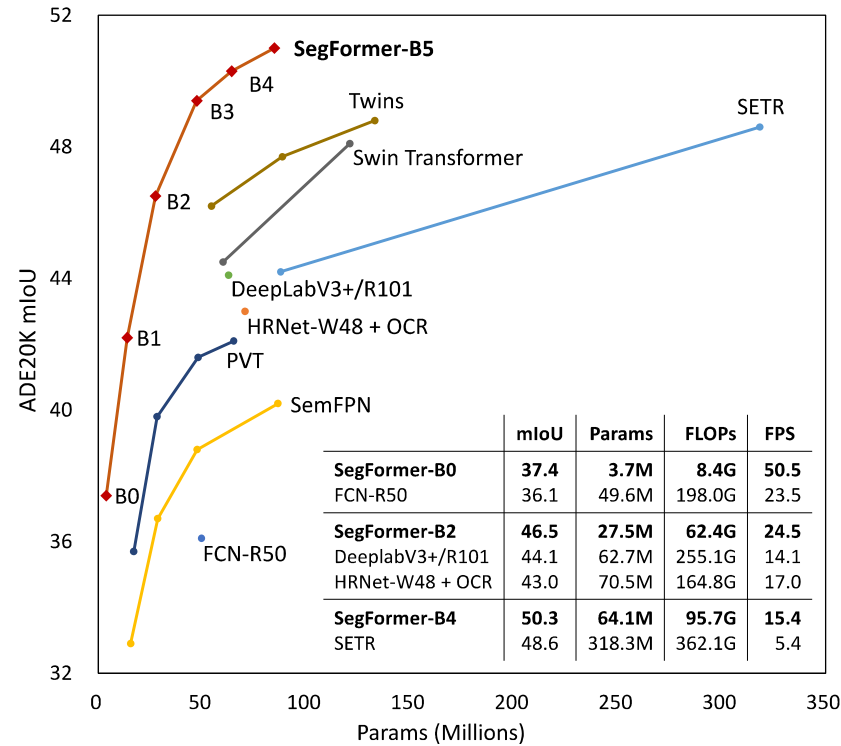
* CNNs

Transformer for Detection: DETR



Predict either object or no object labels at for boxes/regions

Transformer for Segmentation: Segformer



SegFormer

SETR

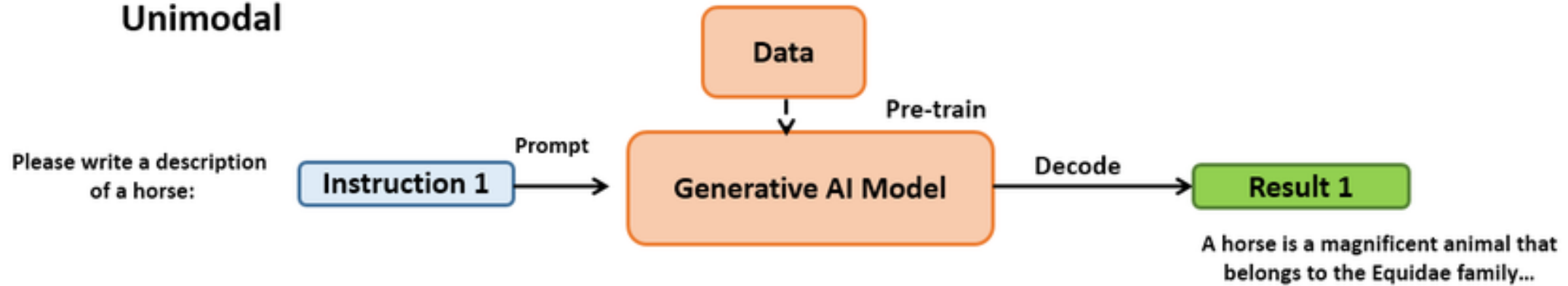
DeepLabV3+

Transformers for ALL

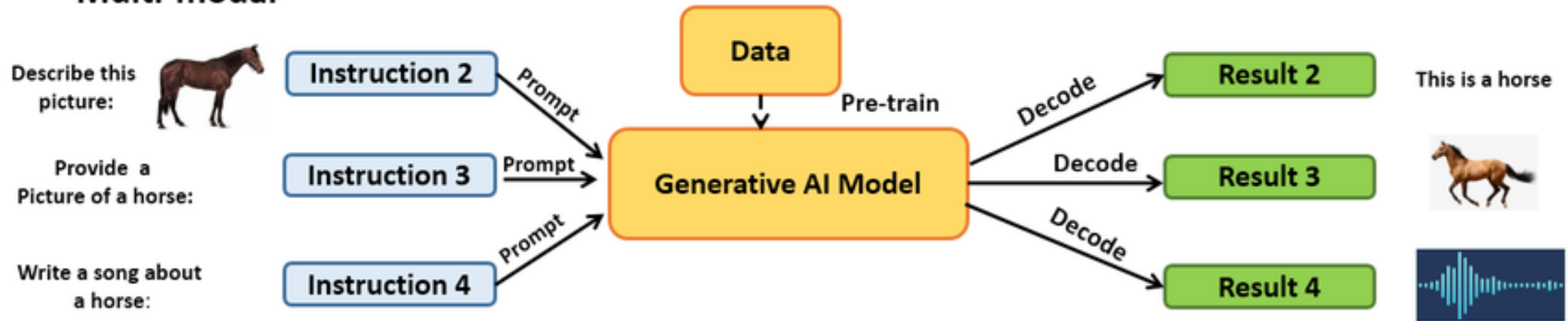
- Transformers have become the default architecture for a number of text/language processing
 - Text classification
 - Machine translation
 - Question answering
- It is now extended for
 - Speech (Eg. Speech transformer (2018), Tera (2021))
 - Vision (Eg. ViT)
 - Graphs (Eg. Graphormer(2021))
 - Multimodal Tasks (Eg. VQA)
 - Etc.

Multimodality

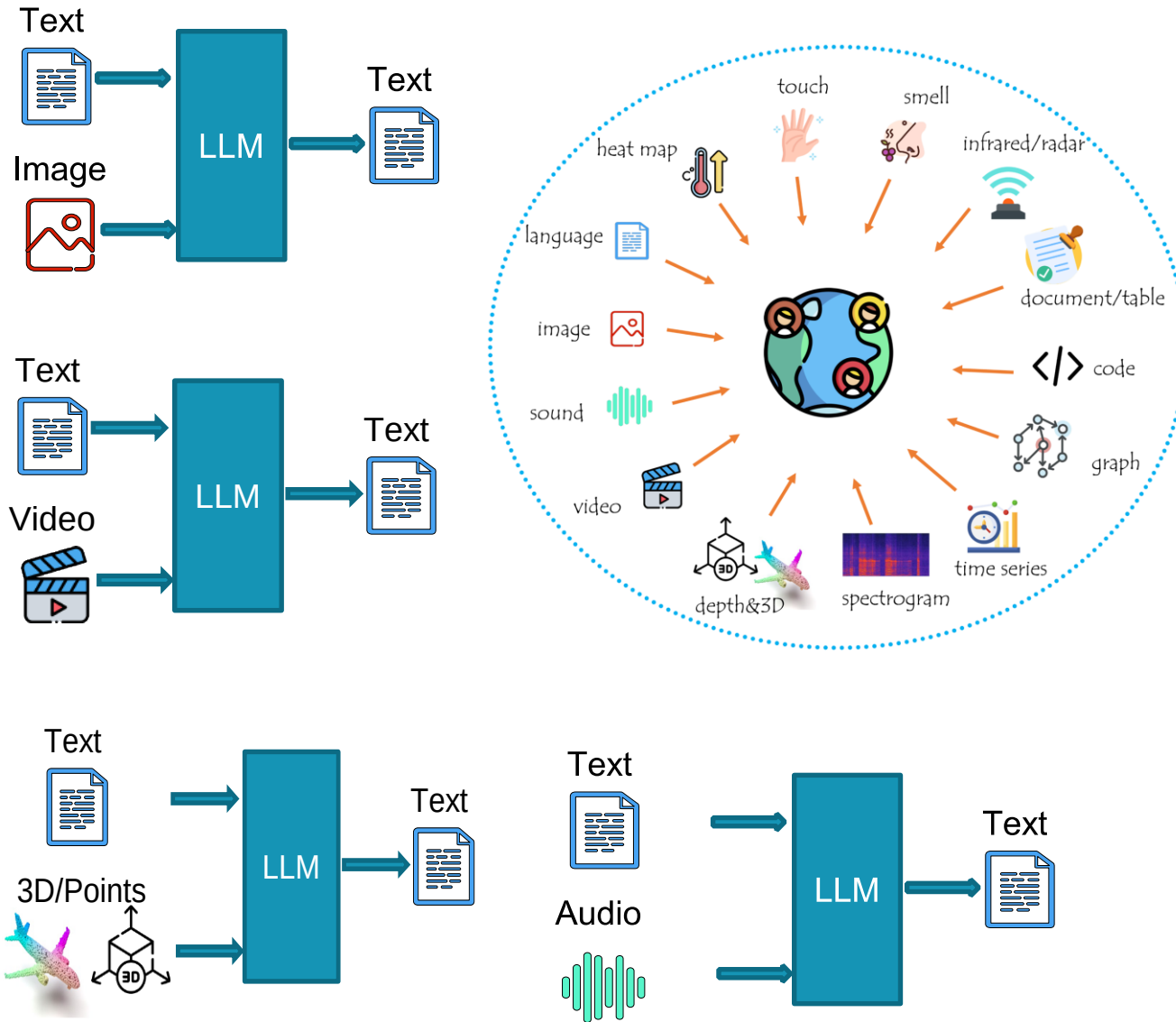
Unimodal



Multi-modal

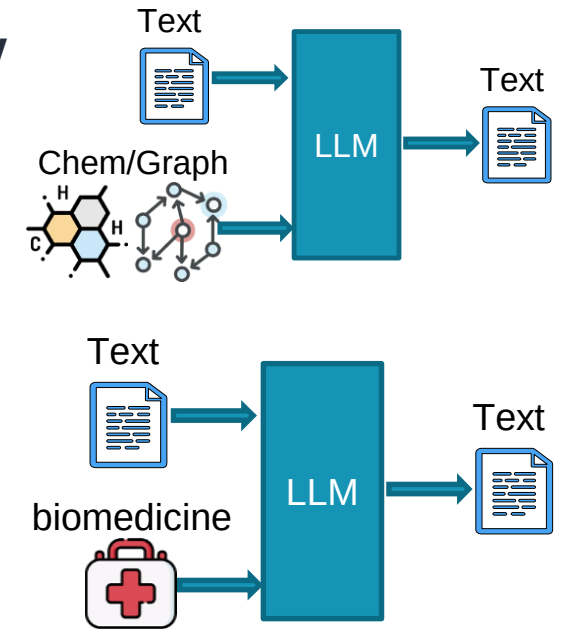


Multimodality



Molecule & Chemistry

- ChemGPT
- SPT
- T5 Chem
- ChemLLM
- MolCA
- MolXPT
- MolSTM
- GIMLET



Bio-/Medical & Healthcare

- | | | |
|--|--|--|
| <ul style="list-style-type: none"> BioGPT DrugGPT BioMedLM OphGLM GatorTron GatorTronGPT MEDITRON | <ul style="list-style-type: none"> DoctorGLM BianQue ClinicalGPT Qilin-Med ChatDoctor BenTsao HuatuoGPT | <ul style="list-style-type: none"> MedAlpaca AlpaCare Zhongjing PMC-LLaMA CPLLM MedPaLM 2 BioMedGPT |
|--|--|--|

Summary

- Transformers have evolved as a powerful architecture
 - For sequence processing tasks
 - And now for many other tasks (including vision)
- Sequence to sequence was the primary task.
 - Now adapting to other modalities
 - Only encode/decoder and both
- Core idea of
 - Attention
 - Pretraining on Large unlabeled data
- Easy to work with
 - Parallelization
 - Ease in taking care of long range dependencies, Larger context

Thanks!!

Questions?